

Relatório – Sistema de Vendas e Fidelidade

Cafeteria Geek “Byte & Brew”

1. Introdução

Este trabalho tem como objetivo aplicar os conceitos fundamentais da Programação Orientada a Objetos (POO) por meio do desenvolvimento de um sistema de vendas e fidelidade para a cafeteria temática **Byte & Brew**.

O sistema simula o funcionamento de uma cafeteria geek, contemplando o gerenciamento de produtos, clientes, pedidos, controle de estoque, aplicação de promoções e um programa de fidelidade baseado em XP (Experiência).

Durante o desenvolvimento foram aplicados conceitos como herança, polimorfismo, encapsulamento, abstração, interfaces, exceções personalizadas e organização em pacotes.

2. Objetivos do Sistema

O sistema foi desenvolvido com os seguintes objetivos:

- Gerenciar produtos do cardápio (comidas e bebidas)
- Controlar o estoque dos produtos
- Cadastrar e gerenciar clientes
- Registrar pedidos de compra
- Aplicar regras de fidelidade (XP)
- Implementar promoções em eventos especiais
- Garantir consistência com validações e exceções

3. Estrutura do Projeto

O projeto foi organizado em pacotes, seguindo boas práticas de desenvolvimento:

`br.edu.cafeteria`

```
|— app
|— modelo
|— servico
|— excecao
```

Responsabilidade dos pacotes:

- **app**: classe principal (execução do sistema)
- **modelo**: entidades do sistema (Produto, Cliente, Pedido etc.)
- **servico**: regras de negócio e promoções
- **excecao**: tratamento de erros personalizados

4. Modelo de Classes (Visão Geral)

O sistema é composto pelos seguintes principais componentes:

- Produto (abstrato)
 - Comida
 - Bebida
- Cliente (abstrato)
 - Cliente Standard
 - Cliente VIP
- Pedido
- ItemPedido
- Atendente
- Interface Promocional
- Promoção de Evento Geek

5. Diagramas e Relacionamentos

5.1 Herança

O sistema utiliza herança para especializar entidades:

- Produto → Comida / Bebida
- Cliente → ClienteStandard / ClienteVIP

5.2 Associação entre Classes

- Um Pedido possui vários ItemPedido
- Um ItemPedido referencia um Produto
- Um Pedido está associado a um Cliente (ou cliente casual)
- Um Pedido está associado a um Atendente

5.3 Polimorfismo

Foi aplicado de diferentes formas:

Polimorfismo por inclusão

Produtos tratados de forma genérica dentro de pedidos.

Polimorfismo por sobrescrita

O método `calcularXP()` é sobrescrito nas classes:

- `ClienteStandard`
- `ClienteVIP`

Polimorfismo por interface

A interface `Promocional` permite diferentes tipos de promoção.

5.4 Interface

Foi criada a interface:

`Promocional`

Responsável por definir o comportamento de descontos em promoções.

A classe `PromocaoEventoGeek` implementa essa interface.

6. Regras de Negócio

6.1 Controle de Estoque

Um produto não pode ser vendido se não houver quantidade suficiente em estoque.

Isso é garantido pela exceção:

- `EstoqueInsuficienteException`

6.2 Programa de Fidelidade (XP)

O sistema possui dois tipos de clientes:

Cliente Standard

- Ganha 1 XP por R\$ 1,00 gasto

Cliente VIP

- Ganha 2 XP por R\$ 1,00 gasto
- Pode pagar compras com XP
- Conversão: 10 XP = R\$ 1,00

6.3 Resgate de XP

O Cliente VIP pode utilizar XP para pagar compras, desde que possua saldo suficiente.

Caso contrário, é lançada a exceção:

- `PontosInsuficientesException`

6.4 Promoções

O sistema suporta promoções através da interface `Promocional`.

Durante eventos especiais, é aplicada:

- 10% de desconto em bebidas

7. Tratamento de Exceções

Foram criadas exceções personalizadas para garantir consistência do sistema:

`EstoqueInsuficienteException`

Lançada quando o estoque não é suficiente para atender a venda.

`PontosInsuficientesException`

Lançada quando o cliente não possui XP suficiente para pagamento.

Essas exceções são do tipo **checked**, garantindo tratamento obrigatório.

8. Conceitos de Programação Orientada a Objetos

8.1 Encapsulamento

Todos os atributos das classes são privados, com acesso controlado por métodos.

8.2 Herança

Utilizada para reaproveitamento e especialização de classes.

8.3 Polimorfismo

Aplicado em:

- métodos sobrescritos (`calcularXP`)

- uso de interface (Promocional)
- manipulação genérica de produtos

8.4 Abstração

Classes como Produto e Cliente são abstratas, representando conceitos genéricos.

8.5 Interfaces

Permitem extensibilidade do sistema sem modificar classes existentes.

9. Fluxo do Sistema

O funcionamento geral do sistema segue as etapas:

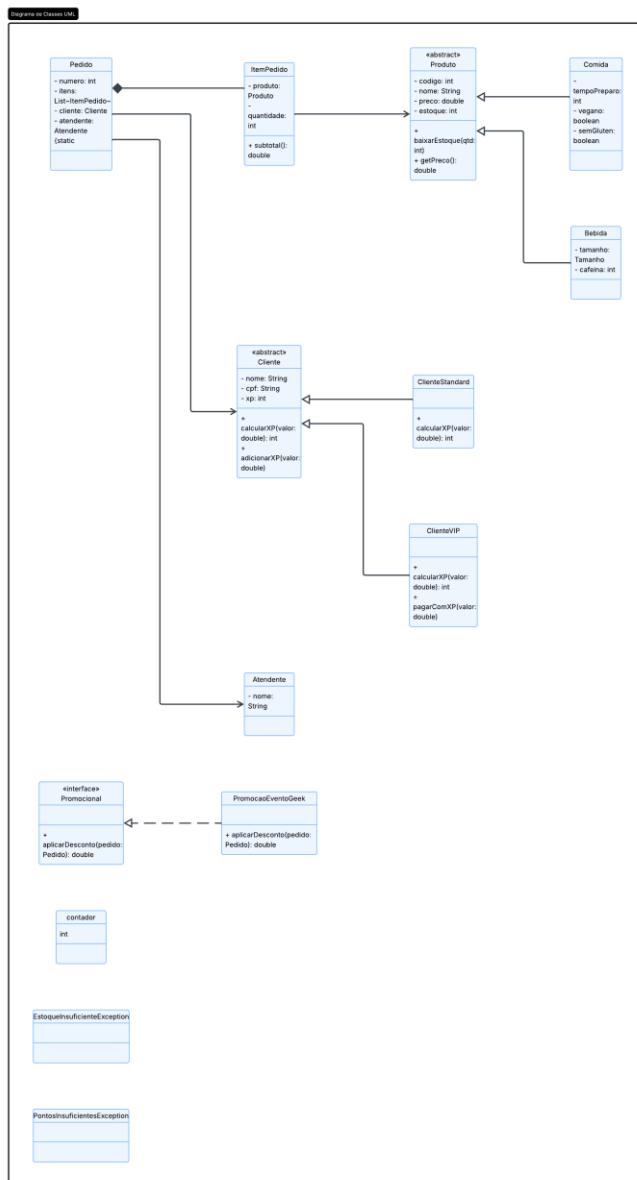
1. Cadastro de produtos
2. Cadastro de clientes
3. Abertura de pedido
4. Adição de itens ao pedido
5. Verificação de estoque
6. Cálculo do total
7. Aplicação de promoções
8. Finalização da venda
9. Atualização de XP e estoque

10. Conclusão

O sistema desenvolvido para a cafeteria **Byte & Brew** permite a aplicação prática dos principais conceitos de Programação Orientada a Objetos.

A solução demonstra organização modular, reutilização de código e boa separação de responsabilidades.

Além disso, o uso de herança, polimorfismo, interfaces e exceções personalizadas garante um sistema flexível, extensível e alinhado com boas práticas de desenvolvimento em Java.



Produto.java

```
package br.edu.cafeteria.modelo;
```

```
public abstract class Produto {
```

```
    private int codigo;
    private String nome;
    private double preco;
    private int estoque;
```

```
    public Produto(int codigo, String nome, double preco, int
estoque) {
        this.codigo = codigo;
```

```

        this.nome = nome;
        this.preco = preco;
        this.estoque = estoque;
    }

    public int getCodigo() { return codigo; }
    public String getNome() { return nome; }
    public double getPreco() { return preco; }
    public int getEstoque() { return estoque; }

    public void baixarEstoque(int qtd) {
        this.estoque -= qtd;
    }
}

```

Comida.java

```

package br.edu.cafeteria.modelo;

public class Comida extends Produto {

    private int tempoPreparo;
    private boolean vegano;
    private boolean semGluten;

    public Comida(int codigo, String nome, double preco, int
estoque,
                    int tempoPreparo, boolean vegano, boolean
semGluten) {
        super(codigo, nome, preco, estoque);
        this.tempoPreparo = tempoPreparo;
        this.vegano = vegano;
        this.semGluten = semGluten;
    }
}

```

Bebida.java

```

package br.edu.cafeteria.modelo;

public class Bebida extends Produto {

```

```

    private Tamanho tamanho;
    private int cafeina;

    public Bebida(int codigo, String nome, double preco, int
estoque,
                Tamanho tamanho, int cafeina) {
        super(codigo, nome, preco, estoque);
        this.tamanho = tamanho;
        this.cafeina = cafeina;
    }
}

```

Tamanho.java

```

package br.edu.cafeteria.modelo;

public enum Tamanho {
    P, M, G
}

```

Cliente.java

```

package br.edu.cafeteria.modelo;

public abstract class Cliente {

    protected static final int CONVERSAO_XP = 10;

    private String nome;
    private String cpf;
    protected int xp;

    public Cliente(String nome, String cpf) {
        this.nome = nome;
        this.cpf = cpf;
    }

    public abstract int calcularXP(double valor);

    public void adicionarXP(double valor) {

```



```

        xp += calcularXP(valor);
    }

    public int getXp() {
        return xp;
    }
}

```

ClienteStandard.java

```

package br.edu.cafeteria.modelo;

public class ClienteStandard extends Cliente {

    public ClienteStandard(String nome, String cpf) {
        super(nome, cpf);
    }

    @Override
    public int calcularXP(double valor) {
        return (int) valor;
    }
}

```

ClienteVIP.java

```

package br.edu.cafeteria.modelo;

import br.edu.cafeteria.excecao.PontosInsuficientesException;

public class ClienteVIP extends Cliente {

    public ClienteVIP(String nome, String cpf) {
        super(nome, cpf);
    }

    @Override
    public int calcularXP(double valor) {
        return (int) (valor * 2);
    }
}

```

```

        public void pagarComXP(double valor) throws
PontosInsuficientesException {
            int necessario = (int) (valor * CONVERSAO_XP);

            if (xp < necessario) {
                throw new PontosInsuficientesException("XP
insuficiente");
            }

            xp -= necessario;
        }
    }
}

```

Atendente.java

```

package br.edu.cafeteria.modelo;

public class Atendente {

    private String nome;

    public Atendente(String nome) {
        this.nome = nome;
    }

    public String getNome() {
        return nome;
    }
}

```

ItemPedido.java

```

package br.edu.cafeteria.modelo;

public class ItemPedido {

    private Produto produto;
    private int quantidade;

    public ItemPedido(Produto produto, int quantidade) {
        this.produto = produto;
    }
}

```

```

        this.quantidade = quantidade;
    }

    public double subtotal() {
        return produto.getPreco() * quantidade;
    }

    public Produto getProduto() {
        return produto;
    }
}

```

Pedido.java

```

package br.edu.cafeteria.modelo;

import java.util.ArrayList;
import java.util.List;
import br.edu.cafeteria.servico.Promocional;

public class Pedido {

    private static int contador = 1;

    private int numero;
    private List<ItemPedido> itens = new ArrayList<>();
    private Cliente cliente;
    private Atendente atendente;

    public Pedido(Cliente cliente, Atendente atendente) {
        this.numero = contador++;
        this.cliente = cliente;
        this.atendente = atendente;
    }

    public void adicionarItem(Produto p) {
        adicionarItem(p, 1);
    }

    public void adicionarItem(Produto p, int qtd) {
        itens.add(new ItemPedido(p, qtd));
    }
}

```

```

    public double calcularTotal() {
        double total = 0;
        for (ItemPedido i : itens) {
            total += i.subtotal();
        }
        return total;
    }

    public double calcularTotal(Promocional promo) {
        return calcularTotal() - promo.aplicarDesconto(this);
    }

    public List<ItemPedido> getItens() {
        return itens;
    }
}

```

SERVIÇO

Promocional.java

```

package br.edu.cafeteria.servico;

import br.edu.cafeteria.modelo.Pedido;

public interface Promocional {
    double aplicarDesconto(Pedido pedido);
}

```

PromocaoEventoGeek.java

```

package br.edu.cafeteria.servico;

import br.edu.cafeteria.modelo.*;

public class PromocaoEventoGeek implements Promocional {

    @Override
    public double aplicarDesconto(Pedido pedido) {

```

```

        double desconto = 0;

        for (ItemPedido item : pedido.getItems()) {
            if (item.getProduto() instanceof Bebida) {
                desconto += item.subtotal() * 0.10;
            }
        }

        return desconto;
    }
}

```

EXCEÇÃO

EstoqueInsuficienteException.java

```
package br.edu.cafeteria.excecao;
```

```

public class EstoqueInsuficienteException extends Exception {
    public EstoqueInsuficienteException(String msg) {
        super(msg);
    }
}

```

PontosInsuficientesException.java

```
package br.edu.cafeteria.excecao;
```

```

public class PontosInsuficientesException extends Exception {
    public PontosInsuficientesException(String msg) {
        super(msg);
    }
}

```

Main.java

```
package br.edu.cafeteria.app;

import br.edu.cafeteria.modelo.*;
import br.edu.cafeteria.servico.PromocaoEventoGeek;

public class Main {

    public static void main(String[] args) {

        Produto cafe = new Bebida(1,"Cafe",10,10,Tamanho.M,100);
        Produto bolo = new Comida(2,"Bolo",20,5,30,false,false);

        Cliente c = new ClienteStandard("Ana","123");

        Atendente a = new Atendente("Joao");

        Pedido p = new Pedido(c,a);

        p.adicionarItem(cafe,2);
        p.adicionarItem(bolo,1);

        PromocaoEventoGeek promo = new PromocaoEventoGeek();

        System.out.println("Total: " + p.calcularTotal());
        System.out.println("Com desconto: " +
p.calcularTotal(promo));
    }
}
```